What is JWT?

JSON Web Token (JWT) is a **secure method to transmit information between client and server** in the form of a **digitally signed token**.

It is commonly used for **authentication and authorization** in modern web applications, including the **MERN stack**.

JWT allows a user to **log in once and access protected routes without repeatedly sending credentials**.

Structure of JWT

A JWT has **3 parts separated by dots**

header.payload.signature

Header

Contains the algorithm and token type.

Example

```
{
 "alg": "HS256",
 "typ": "JWT"
}
```

Payload

Contains the **data (claims)**.

Example

```
{
 "userId": "12345",
 "email": "user@mail.com"
}
```

Signature

Used to **verify the token authenticity**.

```
HMACSHA256(
 base64UrlEncode(header) + "." +
 base64UrlEncode(payload),
 secret_key
)
```

How JWT Works in MERN

Process:

User Login
↓
Server verifies credentials
↓
Server generates JWT
↓
JWT sent to client
↓
Client stores token (localStorage/cookie)
↓
Client sends token with API request
↓
Server verifies token
↓
Access granted

Example Request

Authorization: Bearer TOKEN

MERN JWT Authentication Architecture

React (Frontend)
↓
Login Request
↓
Node + Express API
↓
JWT Created
↓
MongoDB User Data

Technologies used

- MongoDB
- Express.js
- React
- Node.js
- jsonwebtoken

Complete MERN JWT Authentication Example

Backend (Node + Express)

Install dependencies

```
npm init -y
npm install express mongoose cors jsonwebtoken bcryptjs
```

---

server.js

```
const express = require("express")
const mongoose = require("mongoose")
const cors = require("cors")

const authRoutes = require("./routes/auth")

const app = express()

app.use(express.json())
app.use(cors())

mongoose.connect("mongodb://localhost:27017/jwtmern")

app.use("/api", authRoutes)

app.listen(5000, () => {
 console.log("Server running on port 5000")
})
```

User Model

models/User.js

```
const mongoose = require("mongoose")

const UserSchema = new mongoose.Schema({
 name: String,
 email: String,
 password: String
})

module.exports = mongoose.model("User", UserSchema)
```

JWT Middleware

middleware/auth.js

```
const jwt = require("jsonwebtoken")

module.exports = function(req,res,next){

 const token = req.header("Authorization")

 if(!token){
  return res.status(401).json({msg:"No token"})
 }

 try{
```

```
  const decoded = jwt.verify(token,"secret123")

  req.user = decoded

  next()

 }catch(err){
  res.status(401).json({msg:"Token invalid"})
 }

}
```

Authentication Routes

routes/auth.js

```
const express = require("express")
const router = express.Router()
const bcrypt = require("bcryptjs")
const jwt = require("jsonwebtoken")

const User = require("../models/User")
const auth = require("../middleware/auth")

// Register

router.post("/register", async(req,res)=>{

 const {name,email,password} = req.body

 const hashed = await bcrypt.hash(password,10)

 const user = new User({
  name,
  email,
  password: hashed
 })

 await user.save()

 res.json({msg:"User Registered"})
})


// Login

router.post("/login", async(req,res)=>{

 const {email,password} = req.body

 const user = await User.findOne({email})

 if(!user){
  return res.status(400).json({msg:"User not found"})
 }
```

```
 const isMatch = await bcrypt.compare(password,user.password)

 if(!isMatch){
  return res.status(400).json({msg:"Wrong password"})
 }

 const token = jwt.sign(
  {id:user._id},
  "secret123",
  {expiresIn:"1h"}
 )

 res.json({token})

})


// Protected Route

router.get("/profile", auth, async(req,res)=>{

 const user = await User.findById(req.user.id).select("-password")

 res.json(user)

})

module.exports = router
```

React Frontend

Install dependencies

```
npx create-react-app client
cd client
npm install axios
```

Login Component

```
import React,{useState} from "react"
import axios from "axios"

function Login(){

 const [email,setEmail] = useState("")
 const [password,setPassword] = useState("")

 const login = async()=>{

  const res = await axios.post("http://localhost:5000/api/login",{
   email,
   password
  })
```

```
 localStorage.setItem("token",res.data.token)
}

return(

 <div>

  <input placeholder="email"
  onChange={e=>setEmail(e.target.value)} />

  <input placeholder="password"
  type="password"
  onChange={e=>setPassword(e.target.value)} />

  <button onClick={login}>Login</button>

 </div>

)

}

export default Login
```

Access Protected Route

```
const token = localStorage.getItem("token")

axios.get("http://localhost:5000/api/profile",{
 headers:{
  Authorization: token
 }
})
```

Database Example

MongoDB Document

```
{
 "_id":"123",
 "name":"John",
 "email":"john@mail.com",
 "password":"hashed_password"
}
```